

Demystifying WORKLOAD System Statistics Gathering

You must have system statistics if you want the optimizer to evaluate correctly the cost of full table scans vs. index access. But you have two options to gather them: NOWORKLOAD and WORKLOAD. The latter seems to be the more accurate one because it fits your workload. But do you really know how they are calculated?

Franck Pachot, Senior Consultant, dbi services ♠

When I recommend gathering system statistics, I always advise checking them in order to avoid weird values (query here: <http://goo.gl/rhyRqA>). Reasons for weird values can be: bug, non-representative workload, filesystem cache effects, any smart optimisation done behind the scene by the storage, etc.

Another reason is the direct-path reads introduced in 11g by serial direct-path reads. They do multiblock reads, but are not taken into account by some system statistics. Let's get deeper and understand how WORKLOAD statistics are gathered.

The Test Case

On a test database that has no activity, I'm creating a 1000 block table. My goal is to gather WORKLOAD system statistics during a simple table full scan on that table, and see how it calculates SREADTIM, MREADTIM and MBRC.

```
SQL> connect demo/demo
Connected.

SQL> drop table DEMO;
Table dropped.

SQL> create table DEMO pctfree 99 as select rpad('x',1000,'x')
n from dual connect by level <=1000;
Table created.
```

Then I run a simple select between the calls to 'start' and

'stop' procedures of the dbms_stats WORKLOAD system stats gathering.

```
SQL> exec dbms_stats.gather_system_stats('start');
PL/SQL procedure successfully completed.
```

```
SQL> connect demo/demo
Connected.
```

```
SQL> select count(*) from DEMO;
```

```

COUNT (*)
-----
1000
```

I check the physical read statistics (this is why I have reconnected my session so that I can query v\$mystat without doing the delta).

```
SQL> select name,value from v$mystat join v$statname
using(statistic#) where name like 'phy%' and value>0;
```

NAME	VALUE
physical reads	1000
physical reads direct	1000
physical read IO requests	23

I've read 1000 blocks in 23 i/o calls (thanks to my multiblock read count).

I stop my WORKLOAD statistics gathering:

```
SQL> exec dbms_stats.gather_system_stats('stop');
PL/SQL procedure successfully completed.
```

And check the system statistics that have been set:

```
SQL> select * from sys.aux_stats$;
```

SNAME	PNAME	PVAL1
SYSSTATS_INFO	STATUS	
SYSSTATS_INFO	DSTART	
SYSSTATS_INFO	DSTOP	
SYSSTATS_INFO	FLAGS	1
SYSSTATS_MAIN	CPUSPEEDNW	2715
SYSSTATS_MAIN	IOSEEKTIM	10
SYSSTATS_MAIN	IOTFRSPEED	4096
SYSSTATS_MAIN	SREADTIM	
SYSSTATS_MAIN	MREADTIM	4.708
SYSSTATS_MAIN	CPUSPEED	2715
SYSSTATS_MAIN	MBRC	
SYSSTATS_MAIN	MAXTHR	
SYSSTATS_MAIN	SLAVETHR	

I have no SREADTIM because I didn't make any single block read. So it is not set and the CBO will estimate it from IOSEEKTIM+block_size/IOFRSPEED.

I have MREADTIM because I did enough multiblock reads and they have been timed. 4 milliseconds seems to be accurate for my VM on my laptop.

But MBRC is not set. It should have been set in WORKLOAD mode and I've made multiblock reads. But there is a reason for that, which can be guessed from the above statistics. I've made only 'physical reads direct' and we will see later that system statistics do not account for the multiblock reads that are done in direct-path i/o.

Tracing dbms_stats

Yes we can trace dbms_stats. It's not sql_trace. It's only some debug messages showing intermediate steps or values. I've seen that referenced for the first time by the Pythian blog (www.pythian.com/blog/options-for-tracing-oracle-dbms_stats/).

I set the flags '1' in order to have the messages with dbms_output (and I set serveroutput on) and the '262144' which traces the system statistics gathering (this is new in 12.1.0.2 and was not yet documented in the Pythian blog).

```
SQL> exec dbms_stats.set_global_prefs('TRACE',1+262144);
PL/SQL procedure successfully completed.

SQL> set serveroutput on
```

And I do the same as before:

```
SQL> exec dbms_stats.gather_system_stats('start');
```

```
DBMS_STATS: System Stats Information:
reason is gather_system_stats
start option
target is dict
flag is 51
stattab: statid: statown:
DEMO

Infostats:
```

```
status: MANUALGATHERING, dstart: 01-22-2015 17:21,
dstop: 01-22-2015 17:21, flags: 1
MainStats:
cpuspeednw: , ioseektim: ,
iotfrspeed: , cpuspeed: , sreadtim: , mreadtim: , mbr: ,
maxthr: ,
slavethr:
TempStats:
sblkcrds: 15586, sblkrdtim: 28151.772, mblkcrds: 2030,
mblkrdtim: 3673.889, mbrtotal: 34254, cpucycles: 76692, cputim:
28245, job: 0, cache_job: 758

PL/SQL procedure successfully completed.
```

Lots of information came through dbms_output. I run my full table scan and check that I've the same session statistics about the physical reads I've done:

```
SQL> connect demo/demo
Connected.

SQL> select count(*) from DEMO;
```

COUNT(*)
1000

```
SQL> select name,value from v$mystat join v$statname
using(statistic#) where name like 'phy%' and value>0;
```

NAME	VALUE
physical reads	1000
physical reads direct	1000
physical read IO requests	23

And I stop the WORKLOAD statistics gathering:

```
SQL> set serveroutput on
SQL> exec dbms_stats.gather_system_stats('stop');
```

```
DBMS_STATS: System Stats Information:
reason is gather_system_stats stop
option
target is dict
flag is 31
stattab: statid: statown:
DEMO

Infostats:
status: COMPLETED, dstart: 01-22-2015 17:21, dstop:
01-22-2015 17:21, flags: 1
MainStats:
cpuspeednw: 2715, ioseektim: 10,
iotfrspeed: 4096, cpuspeed: 2715, sreadtim: , mreadtim: 1.243,
mbr: ,
maxthr: , slavethr:
TempStats:
sblkcrds: 15586, sblkrdtim: 28151.772, mblkcrds: 2053, mblkrdtim:
3702.479, mbrtotal: 34254, cpucycles: 76692,
cputim: 28245, job: 0, cache_job: 0

PL/SQL procedure successfully completed.
```

The 'tempstats' are the cumulative statistics that are gathered and dbms_stat calculates the difference between the 'stop' and 'start'. Let's do it ourselves.

sblkcrds: 15586-15586=0
is the number of single block reads (the 'db file sequential read'). We have none of them here because my full table scan is doing only multiblock reads.

sblkrdtim: 28151.772-28151.772=0
is the time spent in single block reads

mbldrds: 2053-2030=23

this matches the 'physical read IO requests' above. Those are my multiblock i/o calls.

mbldrtdim: 3702.479-3673.889=28.59 milliseconds
is the time spend in those multiblock reads

mbtrtotal: 34254-34254=0

Here the name is misleading. First because the unit is the number of blocks, but we will see that later. However it's not incremented by every kind of multiblock reads, only those that are going through the buffer cache (the 'db file scattered read').

And, in my case, because I'm reading a large table which has no buffers in cache, Oracle is using the serial direct-path reads available from 11g. That's why we have direct-path reads instead of 'db file scattered read'.

So we have a clue here about the reason why MBRC is not set by dbms_stats WORKLOAD statistics. Only the blocks read to buffer cache are counted and I have none of them. I will verify it below, but first let's check at the statistics gathered:

```
SQL> select * from sys.aux_stats$;
```

SNAME	PNAME	PVAL1
SYSSTATS_INFO	STATUS	
SYSSTATS_INFO	DSTART	
SYSSTATS_INFO	DSTOP	
SYSSTATS_INFO	FLAGS	1
SYSSTATS_MAIN	CPUSPEEDNW	2715
SYSSTATS_MAIN	IOSEEKTIME	10
SYSSTATS_MAIN	IOTFRSPED	4096
SYSSTATS_MAIN	SREADTIM	
SYSSTATS_MAIN	MREADTIM	1.243
SYSSTATS_MAIN	CPUSPEED	2715
SYSSTATS_MAIN	MBRC	
SYSSTATS_MAIN	MAXTHR	
SYSSTATS_MAIN	SLAVETHR	

Note that we have already seen those numbers from the 'stop' trace 'MainStats'. MREADTIM has been calculated as 28.59/23=1.243

Without Direct Reads

In order to prove my assumption I disable serial direct path reads:

```
SQL> connect demo/demo
Connected.
```

```
SQL> alter session set "_serial_direct_read"=never;
Session altered.
```

I will reproduce here only the relevant lines from the trace.

```
exec dbms_stats.gather_system_stats('start');
...
TempStats:
  sblkrds: 15586, sblkrdtim: 28151.772, mblkrds: 2053,
  mblkrdtim: 3702.479, mbtrtotal: 34254, cpucycles: 76692, cputim:
  28245, job: 0, cache_job: 760
```

PL/SQL procedure successfully completed.

```
SQL> select count(*) from DEMO;
```

```

COUNT(*)
-----
      1000
```

```
SQL> select name,value from v$mystat join v$statname
using(Statistic#) where name like 'phy%' and value>0;
```

NAME	VALUE
physical reads	1000
physical reads cache	1000
physical read IO requests	30
physical reads cache prefetch	970

8 rows selected.

```
SQL> set serveroutput on
```

```
SQL> exec dbms_stats.gather_system_stats('stop');
```

...

```
TempStats:
  sblkrds: 15586, sblkrdtim: 28151.772, mblkrds: 2083,
  mblkrdtim: 3725.163, mbtrtotal: 35254, cpucycles: 76692,
  cputim: 28245, job: 0, cache_job: 0
```

PL/SQL procedure successfully completed.

Now that's different. No direct-path reads but still multiblock reads.

MREADTIM is calculated as (3725.163-3702.479)/(2083-2053)=22684/30=0.756

And now we have a value for mbtrtotal=35254-34254=1000

My 1000 blocks have been accounted now, in order to calculate the MBRC: 1000/30=33.33

Here are the gathered statistics:

```
SQL> select * from sys.aux_stats$;
```

SNAME	PNAME	PVAL1
SYSSTATS_INFO	STATUS	
SYSSTATS_INFO	DSTART	
SYSSTATS_INFO	DSTOP	
SYSSTATS_INFO	FLAGS	1
SYSSTATS_MAIN	CPUSPEEDNW	2715
SYSSTATS_MAIN	IOSEEKTIME	10
SYSSTATS_MAIN	IOTFRSPED	4096
SYSSTATS_MAIN	SREADTIM	
SYSSTATS_MAIN	MREADTIM	.756
SYSSTATS_MAIN	CPUSPEED	2715
SYSSTATS_MAIN	MBRC	33

So the formula for MBRC, which can be applied to both cases, is: 'physical read cache'/'physical read' but, from what I've seen (with sql_trace), it calculates the MBRTOTAL as 'physical_read'-'physical_read_direct' and then MBRC=(MBRTOTAL-SBLKRDS)/MBLKRDS

The Formulas

Without going into the detail, here is how the values that are shown are calculated by the tracing of 'start' and 'stop':

```
SQL> select
  singleblkcrds SBLKRDS,
  10*singleblkrdtim SBLKRDTIM,
  (phyrds-singleblkcrds) MBLKRDS,
  10*(readtim-singleblkrdtim) MBLKRDTIM,
  (physical_read-physical_read_direct) MBRTOTAL
from (
  select
    sum(singleblkcrds) singleblkcrds
    ,sum(singleblkrdtim) singleblkrdtim
    ,sum(phyrds) phyrds
    ,sum(readtim) readtim
    ,sum(phyblkrd) phyblkrd
  from v$filestat
),
(select value physical_read from v$sysstat where
name='physical reads'),
(select value physical_read_direct from v$sysstat where
name='physical reads direct')
;
  SBLKRDS  SBLKRDTIM    MBLKRDS  MBLKRDTIM  MBRTOTAL
-----
    15586      28160         2083      3730      35254
```

Which matches exactly the latest 'TempStats' above.

The '10x' is because v\$filestat times are in 100th of a second. Note that dbms_stats queries the underlying x\$kcfo which in 11.2 became in microseconds and this had brought a bug from 11.2 to 11.2.0.2 where the times were 10000 times larger.

According to my tests, here is how the system stats are set from that:
 $SREADTIM = SBLKRDTIM / SBLKRDS$ and $MREADTIM = MBLKRDTIM / MBLKRDS$

Once again, from my tests in 11g and 12c, it seems that SREADTIM and MREADTIM are not set when there are less than 10 i/o calls. That makes sense because it is better to use defaults (from IOSEEKTIM and IOTFRSPEED) rather than non-significant values.

And the observed multiblock read count formula is:
 $MBRC = (MBRTOTAL - SBLKRDS) / MBLKRDS$ and this one is set only when we have more than 100 multiblocks read ($MBRTOTAL - SBLKRDS$).

Single Block Read

Finally, I run the same but doing only single block reads by setting db_file_multiblock_read_count to 1. As the previous test has read to buffer cache, I need to flush it in order to do some physical i/o.

It will be done with direct-path reads but that doesn't matter because I will have no multiblock reads anyway.

```
SQL> alter system flush buffer_cache;
System altered.

SQL> connect demo/demo
Connected.
SQL> alter session set db_file_multiblock_read_count=1;
Session altered.

SQL> exec dbms_stats.gather_system_stats('start');
...
TempStats:
  sblkrds: 15597, sblkrdtim: 28154.207, mblkrds:
2083, mblkrdtim: 3725.163, mbtotal: 35265, cpucycles: 76692,
cputim:
28245, job: 0, cache_job: 762

PL/SQL procedure successfully completed.

SQL> select count(*) from DEMO;

  COUNT(*)
-----
        1000

SQL> select name,value from v$mystat join v$statname
using(statistic#) where name like 'phy%' and value>0;

NAME                                     VALUE
-----
physical reads                           1018
physical reads cache                      18
physical reads direct                     1000
physical read IO requests                 1018

7 rows selected.

SQL> set serveroutput on
SQL> exec dbms_stats.gather_system_stats('stop');
...
MainStats:
cpuspeednw: 2715, ioseektim: 10,
iotfrspeed: 4096, cpuspeed: 2715, sreadtim: .244, mreadtim: ,
mbrc: ,
maxthr: , slavethr:
TempStats:
  sblkrds: 16706, sblkrdtim: 28425.247,
mblkrds: 2083, mblkrdtim: 3725.163, mbtotal: 35374, cpucycles:
76692,
cputim: 28245, job: 0, cache_job: 0
```

That time I don't query sys.aux_stats\$ because I know I've the values above in 'MainStats'.

Because all reads are single block, only SREADTIM has been calculated and the formula above is verified: $(28425.247 - 28154.207) / (16706 - 15597) = 0.244$

Physical Read Total Multiblock Requests

I have deliberately hidden the 'physical read total multiblock requests' statistic in the multiblock read tests above. The full result in the second test was:

```
SQL> select name,value from v$mystat join v$statname
using(statistic#) where name like 'phy%' and value>0;
```

NAME	VALUE
physical read total IO requests	30
physical read total multi block requests	14
physical read total bytes	8192000
physical reads	1000
physical reads cache	1000
physical read IO requests	30
physical read bytes	8192000
physical reads cache prefetch	970

Do you wonder why I've only 14 multiblock requests accounted when we know that we did 30 'db file sequential read'? That statistic – despite its definition – does not count the multiblocks that are smaller than specified (reason can be extent boundary or blocks already in buffer cache). In my case I've created the

table in an auto allocation extent size where the first 16 extent size is 64k which has only 8 blocks (my block size is 8k). Then only $30 - 16 = 14$ have been counted. This is the reason why dbms_stats calculates the number of multiblock reads from v\$sqlstat rather than from instance statistics.

Conclusions

The goal was to understand how dbms_stats gathers WORKLOAD statistics and get the formula which we can use ourselves, for example to get the values from the AWR history in order to put them in a graph and see how they are regular not not. Here is an example of that: www.dbi-services.com/index.php/blog/entry/when-to-gather-workload-system-statistics

Then you can see the results in a graph and finally set relevant statistics manually. You see how it evolves over time and for all instances (note that when you are in RAC dbms_stats will gather stats for your instance only).

Final remark: if you are unsure, default noworkload statistics are probably correct for common cases (assuming that you didn't set db_file_multiblock_read_count). ■



ABOUT THE AUTHOR

Franck Pachot 
Senior Consultant, dbi services

Franck Pachot is a senior consultant at dbi services in Switzerland. He has 20 years of experience in Oracle databases, in all areas from development, data modeling, performance, administration and training. Franck is an Oracle Certified Master and Oracle ACE who tries to leverage knowledge sharing in forums, publications, presentations.



www.linkedin.com/in/franckpachot
@FranckPachot

MEET A VOLUNTEER: ANDREW

Name: Andrew Watkins
Job Title: Assistant Computer Systems Manager
Company: Birkbeck, University of London

Tell us about yourself in 50 words

I have been a Solaris enthusiast since my University days, so being a co-chair of UKOUG Solaris SIG was a natural progression. My day job means I spend too much time supporting operating systems, keeping up to date with a never-ending list of databases and coding.

What is your goal in life?

My work goal is to make life easier for our students. My life goal is to win the lottery and spend my days on the beach with no computers in sight.

What are your interests?

I love sports and, being Welsh, I have a great interest in rugby, but that is not going too well these days. I relish problem solving so I take great pleasure debugging code and applications software.

When and why did you want to become a UKOUG volunteer?

After a bit of arm twisting I joined the Solaris SIG Committee in 2010. It is my way of giving back having attended the SIG for a few years. It's been a great way of making new contacts, although you can lose friends when you keep badgering them to give talks! I've also volunteered for UKOUG Systems events and UKOUG Tech conference committees. I write for the Oracle Scene magazine and present at the SIG.

What does a day of volunteering involve for you?

No two days are alike, from organising the next SIG meeting and locating speakers, to preparing a presentation, sending out announcements and the occasional conference call.



Which volunteer contribution are you most proud of?

It has to be my involvement in the UKOUG Tech conferences where you have the chance to mould a collection of ideas into a full conference. The best thing about volunteering for me is the other volunteers! I get a buzz from their passion for Oracle technology. The Solaris SIGs are rewarding too, you meet a great bunch of people and while sharing a drink you share a good laugh, war stories and, as a bonus, you pick up a lot of technical knowledge.



@awatkins1966
uk.linkedin.com/in/AndrewWatkins1966